PWN 编程基础

在开始学习 PWN 之前，需要具备一些基本的编程与底层知识。对于初学者来说，不需要一开始就深入掌握所有细节，但至少需要能够读懂简单的 C 程序、使用 Python 编写脚本，并理解基本的 x86-64 汇编代码。

本部分主要包括：

- C 语言
- Python
- 汇编语言

C 语言

C 语言是一种通用的高级语言，也是 PWN 中最常见的程序语言之一。

大部分 PWN 题目的目标程序都由 C/C++ 编写，因此学习 PWN 时，至少需要能够读懂常见的 C 语言代码。

对于初学者来说，不需要先系统学完整本 C 语言教材再开始 PWN，但建议了解：

- 基本变量与数据类型
- 数组与字符串
- 函数
- 指针
- 结构体
- 动态内存分配
- 常见 C 标准库函数

其中，**指针、数组、字符串以及内存相关函数**在后续漏洞分析中尤其重要。

例如在 PWN 中经常能够见到：

```
char buf[64];

read(0, buf, 128);
```

理解数组大小、指针和内存布局之后，才能进一步分析这类代码可能存在的问题。

对于 PWN 手来说，还需要逐渐熟悉一些常见函数，例如：

```
read
write
gets
fgets
scanf
printf
strcpy
memcpy
malloc
free
```

这些内容不需要一次全部掌握，可以在后续做题过程中不断积累。

推荐资料

- 浙江大学翁恺老师《程序设计入门——C语言》
- 《C Primer Plus》
- 《C和指针》

Python

Python 是一种非常适合编写脚本的语言。

在 PWN 中，我们通常使用 Python 处理数据，并使用基于 Python 开发的 `pwntools` 与目标程序进行交互、构造 Payload 和编写 Exploit。

因此不需要把 Python 学到非常深入，但至少建议掌握：

- 变量与基本数据类型
- `if` 判断
- `for` / `while` 循环
- 函数
- `list`、`dict`
- 字符串与字节串
- 模块导入
- 基本文件操作

尤其需要注意 Python 中：

```
str
```

和：

```
bytes
```

是不同的数据类型。

Pwntools 与程序进行二进制交互时，大多数情况下使用的是 `bytes`：

```
payload = b"A" * 64
```

而不是：

```
payload = "A" * 64
```

后续编写 PWN Exploit 时，Python 主要承担：

```
与程序交互
    ↓
处理泄漏数据
    ↓
计算地址
    ↓
构造 Payload
    ↓
自动完成漏洞利用
```

推荐资料

- Python 官方文档
- 菜鸟教程 Python3 教程
- 《Fluent Python》

汇编语言

CPU 实际执行的是机器指令，而汇编语言可以使用更加容易理解的符号形式表示这些指令。

PWN 中虽然可以通过 IDA 等工具查看伪代码，但在分析：

- 函数调用
- 栈帧
- 栈溢出
- ROP
- 栈迁移
- Shellcode

等内容时，仍然需要具备基本的汇编阅读能力。

对于初学者来说，并不需要成为 Assembly Master 才能开始学习 PWN。

前期只需要先理解：

- 常用寄存器
- 数据传输
- 栈操作
- 函数调用

- 条件跳转

即可。

常见寄存器

在 x86-64 程序中经常会看到：

寄存器	常见作用
`RAX`	返回值
`RDI`	第一个函数参数
`RSI`	第二个函数参数
`RDX`	第三个函数参数
`RCX`	第四个函数参数
`RSP`	栈顶指针
`RBP`	栈帧基址
`RIP`	当前执行指令地址

这些寄存器的作用在后续使用 GDB 调试时也会频繁遇到。

常见指令

数据传输

```
mov dst, src
```

将 `src` 的值赋给 `dst`。

```
push src
```

将数据压入栈中。

```
pop dst
```

从栈中取出数据。

算术与逻辑

```
add dst, src
sub dst, src
xor dst, src
and dst, src
or dst, src
```

分别对应常见的加、减以及按位逻辑运算。

比较

```
cmp src1, src2
```

比较两个操作数，并设置 CPU 状态标志位。

```
test src1, src2
```

进行按位与运算并设置状态标志位，但不会保存运算结果。

跳转

```
jmp addr
```

无条件跳转。

常见条件跳转包括：

```
je / jz
jne / jnz
ja
jb
jg
jl
jge
jle
```

它们通常配合：

```
cmp
test
```

使用。

函数调用

```
call addr
```

调用函数，并将返回地址压入栈中。

```
ret
```

从栈中取出返回地址，并跳转到该地址继续执行。

理解 `call` 和 `ret` 是后续学习栈溢出和 ROP 的基础。

其他

```
lea dst, src
```

获取地址。

```
syscall
```

进入内核执行系统调用。

```
nop
```

空指令，不执行实际操作。

学习建议

初学阶段优先**认识**：

```
mov
push
pop
lea
cmp
test
jmp
call
ret
```

并结合 IDA 和 GDB 实际观察这些指令的执行效果。

后续学习栈溢出、ROP、Shellcode 等内容时，再**逐步补充更加完整的汇编知识。**

推荐资料

- 王爽《汇编语言》
- x86 / x86-64 汇编相关入门教程
- Intel x86-64 指令手册

FLAG

```
D1NO{c0d1ng_1s_wh3r3_pwn_b3g1ns}
```